SETI – B2 Module – Exam 2022/2023

Asma Smaoui & Ansgar Radermacher
{first-name}.{last-name}@cea.fr

Drone system: navigation system

1 Context

We focus on the modeling of a drone that transports a parcel from a source to a destination, similar as a use case that has been studied in the context of the European project Comp4Drones.

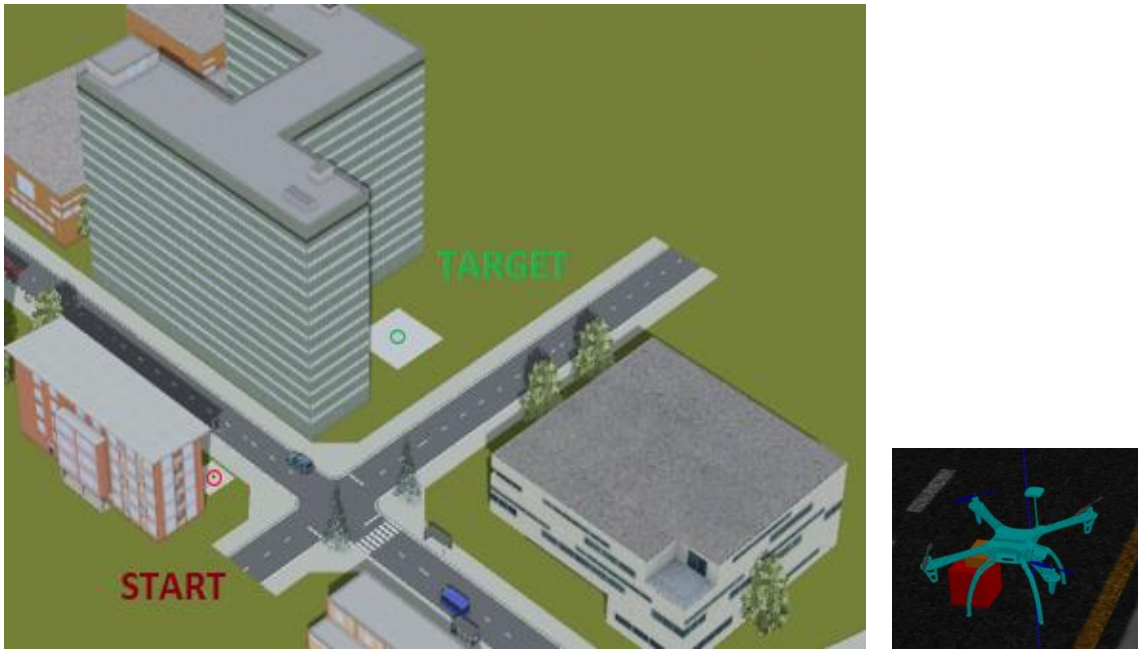


Figure 1: Sample scenario and drone (in simulator)

The high-level operation of the drone is controlled by a mission-management module that takes care of the different phases of the drone mission, such as flying to the start place, landing there and picking up the parcel.

The navigation module has an operation to set a target position (GPS coordinates). It then calculates a trajectory to this position that takes into account statically known obstacles such as higher buildings and no-fly zones (geo-fencing). The navigation also relies on a perception module that is able to detect obstacles using either a camera or a Lidar[1] system. The camera based perception is based on a Bayesian (probabilistic) neural network that is not only making a prediction, but also associates an uncertainty estimation with the prediction. Of course, it is not your task to develop such a network, but you should model a suitable component-based interface (for simplicity, model continuous/streaming data from sensors via "set" operations that take complete frames as parameters). It is possible that the uncertainty becomes too high, e.g. due to

1 https://en.wikipedia.org/wiki/Lidar

weather condition. In this case, the obstacle detection switches input to the Lidar system (which consumes more energy and is therefore not active by default) or asks an operator to take over. If yes, the operator can directly interact with the control component by sending data containing throttle, yaw, pitch and roll information, as shown in Figure 2.

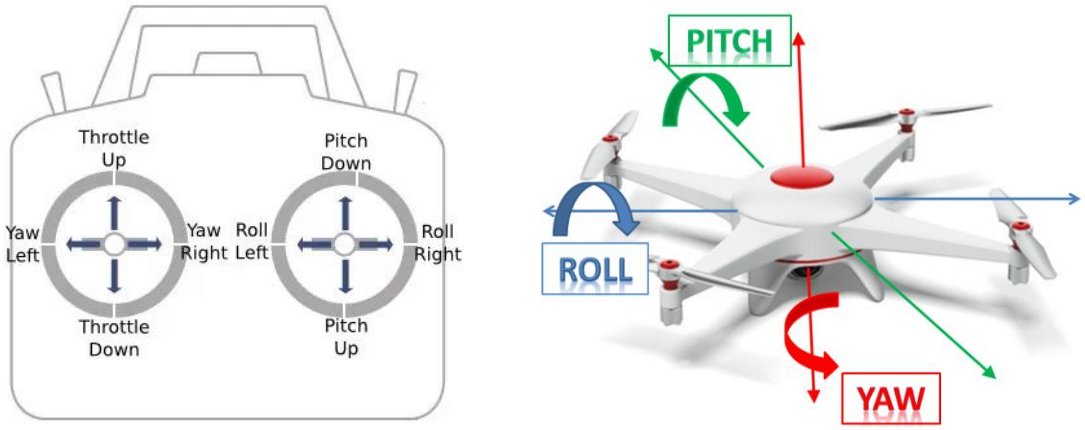


Figure 2: Schematic remote control for throttle, yaw, pitch and roll (source malanghobbies)

2 Exercises

2.1 Functions

In a convenient UML diagram, specify the main system *functions* as well as their relationships and interaction with the environment. Do not exceed 5 external actors (try to choose the most important actors such as obstacles) in order to avoid cluttering the diagram

2.2 Composition

In a convenient UML diagram, specify the composition of the drone *components*. The whole drone system has the following sub-systems: perception, control, mission management, and navigation, There is also a sub-system representing the service side of manual control.

The latter is communicate with a remote control which can take over from the autonomous navigation. It has controllers for throttle, pitch, yaw and roll. From a conceptual viewpoint, the actuators are either interactive with the automatic or manual control system

2.3 Behavior

In a convenient UML Diagram, specify the behavior of several drone components according to their states outlined below. If a state has sub-states, use a suitable way to model this situation. You should create state-machines for the two components described below (not for the other components)

The **drone** itself has the states:

- **Off** – the drone is off
- **Grounded** – the drone has been activated, but is on the ground
- **Flying** – the drone is in the air. Identify **sub-states** for the status of the navigation module: **NavManual**, **NavCamera** (default), **NavLidar**

- **BatteryNominal / BatteryLow** – the battery status, these two states are independent of the others
- **ParcelLoaded / PickupEmpty** – two states representing the parcel-load status, these two states are independent of the others

The **mission-management** component has the following states:

- T**akeoff** – The system is taking off until it reaches a certain minimum height
- **FlyTo** – navigate to a goal destination that is stored within the mission management. Please note that the system has to handle at least a pickup and a destination location.
- **Land** – (precision) landing at a specific location
- **PickupParcel** – the drone has landed close enough to a parcel in order to pick it up
- **DropParcel -** the drone has landed and is dropping the parcel.

Please consider that things could go wrong and the sequencing is not necessary linear (e.g. due to a predicted battery-low condition or an abandoning of the mission).

2.4 Domains specific modeling, profile

A simplified automotive profile has four stereotypes. Model the profile (with suitable OCL constraints) and apply the profile to your system model.

a) «ActiveSensor»: an active sensor is represented by an active class having a classifier behavior.
b) «PassiveSensor»: a passive sensor is represented by a normal class. It must contain at least one port with a provided interface
c) «Controller»: a controller is represented by an active class with a classifier behavior. It must have a classifier behavior and attributes indicating the CPU frequency (as an integer with MHz as unit, alternatively examine MARTE´s value specification language) and the provided memory size (integer in MB or MARTE VSL).
d) «Actuator»: an actuator is represented by a class having at least one port with a provided interface

2.5 Activities

Use a suitable diagram to model the behavior of the navigation module that follows way-points from a trajectory. Use the simplified algorithm outlined below.

Obtain the position of the next way-point. Fly in the direction of this position, if no obstacle has been detected within a certain distance to this trajectory. Otherwise compute a new trajectory that respect the minimal distance to the detected obstacle.